Assignment 1
High-End-Devices
Karthik Ganta -002647311

I have connect my pi over the IP 10.159.87.7

Screenshot for the ifconfig:

Question 1: Provide the output of ifconfig, rustc –version, sudo apt list –installed.

```
pi@pi:~ $ ifconfig
eth0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 2c:cf:67:f6:c9:e5  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 106

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 426  bytes 35667 (34.8 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 426  bytes 35667 (34.8 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.159.87.7  netmask 255.255.255.0  broadcast 10.159.87.255
        inet6 fe80::2152:47be:1cdd:db8  prefixlen 64  scopeid 0x20<link>
        inet6 2607:fb91:de9:afc9:69e5:c660:3264:5edd  prefixlen 64  scopeid 0x0<global>
        ether 2c:cf:67:f6:c9:e6  txqueuelen 1000  (Ethernet)
        RX packets 296  bytes 25559 (24.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 257  bytes 35470 (34.6 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Rust Version:

```
pi@pi:~/High_end_devices/High-end-devices/Homework1 $ rustc --version
rustc 1.63.0
pi@pi:~/High_end_devices/High-end-devices/Homework1 $
```

Rust Installation:

```
libstd/oldstable,now 1.5.4+dfsg2-5 armhf [installed,automatic]
pi@pi:~/High_end_devices/High-end-devices/Homework1 $ apt list --installed | grep -i rust

WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

libstd-rust-1.63/oldstable,now 1.63.0+dfsg1-2+rpi1 armhf [installed,automatic]
libstd-rust-dev/oldstable,now 1.63.0+dfsg1-2+rpi1 armhf [installed,automatic]
rustc/oldstable,now 1.63.0+dfsg1-2+rpi1 armhf [installed]
pi@pi:~/High_end_devices/High-end-devices/Homework1 $
```

Question 2: Write a rust progr
am that allocates six static variables (unsigned 8-bit A, unsigned 16-bit B, unsigned 32-bit C, unsigned 64-bit D, unsigned 128-bit E, unsigned 8-bit F) and
six local variables (unsigned 8-bit a, unsigned 16-bit b, unsigned 32-bit c, unsigned 64-bit
d, unsigned 128-bit e, unsigned 8-bit). Print out the addresses of these variables. Show the
output. Draw conclusions.

The below screenshot provides the code of the Rust program

```rust

/*
    Here the A ,B ,C D, E, F are the static Global variable and
*/
static  A : u8  = 10;
static  B : u16 = 20;
static  C : u32 = 30;
static  D : u64 = 40;
static  E : u128 = 50;
static  F : u8 = 60;


fn main(){
    /*
    Here the a ,b ,c d, e, f are the local variable and
    */
    let a : u8  = 10;
    let b : u16 = 20;
    let c : u32 = 30;
    let d : u64 = 40;
    let e : u128 =50;
    let f : u8 = 60;
    println!("<================== Static Variable in Data Segment ============>");
    println!("Address of the Static Variable A (8 bits) or (1 byte) is  {:p}",&A);
    println!("Address of the Static Variable B (16 bits) or (2 byte) is {:p}",&B);
    println!("Address of the Static Variable C (32 bits) or (4 byte) is {:p}",&C);
    println!("Address of the Static Variable D (64 bits) or (8 byte) is {:p}",&D);
    println!("Address of the Static Variable E (128 bits)or (16 byte)is {:p}",&E);
    println!("Address of the Static Variable F (8 bits)  or (1 byte) is {:p}",&F);

    println!("<================== Local Variable in Data Segment ============>");
    println!("Address of the Static Variable A (8 bits)  or (1 byte)is  {:p}",&a);
    println!("Address of the Static Variable A (16 bits) or (2 byte)is  {:p}",&b);
    println!("Address of the Static Variable A (32 bits) or (4 byte)is  {:p}",&c);
    println!("Address of the Static Variable A (64 bits) or (8 byte)is  {:p}",&d);
    println!("Address of the Static Variable A (128 bits)or (16byte)is  {:p}",&e);
    println!("Address of the Static Variable A (8 bits)  or(1 byte)is   {:p}",&f);
}

```

As mentioned in the comment line's ABCDEF are the global variables and the abcdef variable's are the local variable

Output:

```
pi@pi:~/High_end_devices/High-end-devices/Homework1 $ ./practice
<================== Static Variable in Data Segment ============>
Address of the Static Variable A (8 bits)       or (1 byte) is  0x7fde10
Address of the Static Variable B (16 bits) or (2 byte) is       0x7fde12
Address of the Static Variable C (32 bits) or (4 byte) is       0x7fde14
Address of the Static Variable D (64 bits) or (8 byte) is       0x7fde18
Address of the Static Variable E (128 bits)or (16 byte)is       0x7fde20
Address of the Static Variable F (8 bits)  or (1 byte) is       0x7fde30
<================== Local Variable in Data Segment ============>
Address of the Local Variable a (8 bits)  or (1 byte)is 0xffc5f581
Address of the Local Variable b (16 bits) or (2 byte)is 0xffc5f582
Address of the Local Variable c (32 bits) or (4 byte)is 0xffc5f584
Address of the Local Variable d (64 bits) or (8 byte)is 0xffc5f588
Address of the Local Variable e (128 bits)or (16byte)is 0xffc5f590
Address of the Local Variable f (8 bits)  or(1 byte)is  0xffc5f5a7
pi@pi:~/High_end_devices/High-end-devices/Homework1 $
```

Now the conclusion from the output:

Observation 1: We can see that the variable A which is static global variable with the memory location 0x7fde10in the Data Section

Observation 2: We can see that the variable a which is Local variable with the memory location 0xffc5f581 in the Stack Segment .

Observation 3: Both in the stack section and the Data Section the memory location are byte aligned

for example :

Static Variable A u8 (1 byte) with memory location 0x7fde10 and Static Variable B (2 byte) is 0x7fde12 so the Variable A is occupies 1 byte of space

For Static Variable :

Static Variable A occupies ==> (1 byte) with memory 0x7fde10

Static Variable B occupies ==> (2 byte) with memory 0x7fde12 to 0x7fde14

Static Variable C Occcupies (4 bytes) => with memory 0x7fde14 to 0x7fde18

Static Variable D Occcupies (8 bytes) => with memory 0x7fde18 to 0x7fde20

Static Variable E Occcupies (16 bytes) => with memory 0x7fde20 to 0x7fde30

Static Variable F Occcupies (1 bytes) => with memory 0x7fde30

For Local Variable :

Local Variable a occupies ==> (1 byte) with memory 0xffc5f581

Local Variable B occupies ==> (2 byte) with memory 0xffc5f582 to 0xffc5f584

Static Variable C Occcupies (4 bytes) => with memory 0xffc5f584 to 0xffc5f588

Static Variable D Occcupies (8 bytes) => with memory 0xffc5f588 to 0xffc5f590

Static Variable E Occcupies (16 bytes) => with memory 0xffc5f590 to 0xffc5f5a7

Static Variable F Occcupies (1 bytes) => with memory 0xffc5f5a7

Observation 4: Stack Always grows in the downward Direaction.